

# Demonstration Guide

## Safe Semantic Planner — PCCST503 Assignment 1

### 1. Before you start

```
g++ -O2 -std=c++17 -Wall -Wextra -Wpedantic -o safe_planner safe_planner.cpp
```

Have a terminal at least 40 rows tall so a full grid frame fits without scrolling. Total runtime for the whole demonstration below is about 12 minutes including talking.

### 2. Suggested running order

| # | Command                                                                             | Time  | Point being made                                   |
|---|-------------------------------------------------------------------------------------|-------|----------------------------------------------------|
| 1 | <code>./safe_planner</code><br><code>--demo --slow</code>                           | 4 min | incremental replanning, visually                   |
| 2 | <code>./safe_planner</code><br><code>--tests</code>                                 | 1 min | all six required test cases, each verified optimal |
| 3 | <code>./safe_planner</code><br><code>--sweep</code>                                 | 1 min | the cost/safety trade-off, and its limit           |
| 4 | <code>./safe_planner</code><br><code>--bench</code>                                 | 1 min | scale, and what the heuristic buys                 |
| 5 | <code>./safe_planner</code><br><code>--scaling</code>                               | 2 min | where incremental repair wins and where it loses   |
| 6 | <code>./safe_planner</code><br><code>--solve</code><br><code>warehouse.graph</code> | 1 min | it takes real input, not just built-in tests       |

If time is short, run 1 and 5 only. Those two carry the actual contribution; the rest is verification.

### 3. The live demonstration, frame by frame

`--slow` pauses about 0.9 s between frames. Drop `--slow` if you would rather scroll.

A 26 × 12 lattice, 8-connected. A is the agent, G the goal, X a bad state, # a cell whose transitions have been removed, o the current plan.

#### Frame 1 — initial plan

| ..... |

```
| .....X.....|
| .....X.....X.....|
| .....X.....|
| .Aoo.....G.|
| ...oooooooooooooooooo..|
```

“Corner to corner, 312 states. The planner expands 81 of them in 0.04 ms. Notice it dips one row below the direct line — that is the clearance term keeping it away from the two bad states on the left.”

Cost 23.83, minimum clearance 2.00, zero bad states visited.

### Frame 2 — the agent moves six steps

“The agent walks six steps along its own plan. Watch the expansion count.”

**Zero expansions.** This is the moment to slow down:

“Nothing was recomputed. In D\* Lite the search runs backwards from the goal, so the g-values are costs *to the goal* — they do not depend on where the agent is. Moving the agent shifts the heuristic, and that shift is absorbed by a single running offset called  $k_m$  instead of re-keying the queue. This is exactly why D\* Lite was chosen over LPA\*, which searches forward and would have to discard everything here.”

### Frame 3 — a barrier appears

```
| .....ooooo.....|
| .....o#.....o.....|
| .....o#...X..o.....|
| .....o#.....o.....|
| .....X..o..#.....X..o..|
| .....o..#.....G.|
| .....A.....#.....|
```

“A wall drops across column 13, blocking every transition into and out of ten cells. The planner repairs in 167 expansions, 0.10 ms, and routes over the top. Cost rises from 17.4 to 28.97 — the detour is genuinely expensive, and the planner is telling us so.”

### Frame 4 — a gap opens

```
| .....#.....|
| .....ooo.....|
| .....X.....o#..o...X.....|
| .....o..#....o.....G.|
| .....Aoo...#.....ooooo..|
```

“Now one cell of the barrier is restored. **Fifteen expansions** — and the cost drops from 28.97 to 20.73. A fresh search would have expanded a hundred-odd states to find that. The planner only had to reconsider the small region the change actually touched.”

This is the strongest single frame in the demonstration. Pause on it.

## Frame 5 — the goal moves

“Finally the goal jumps to the opposite corner. 96 expansions. This one is *not* cheap, and the design report is explicit about why: g-values are defined relative to the goal, so a new goal makes every stored value meaningless. What is reused is everything whose construction cost something — the adjacency lists, the embeddings, the clearance field, the heap’s allocated storage. The reset is  $O(n)$  with no reallocation. The brief asks for a revised path ‘without rebuilding all data structures whenever possible’, and that is the distinction we are drawing.”

## Frame 6 — the event log

| event                  | expand | time (ms) | cost  | clearance |
|------------------------|--------|-----------|-------|-----------|
| initial plan           | 81     | 0.0415    | 23.83 | 2.00      |
| after 6 steps          | 0      | 0.0000    | 17.41 | 2.24      |
| repair after barrier   | 167    | 0.0977    | 28.97 | 2.24      |
| repair after gap opens | 15     | 0.0106    | 20.73 | 2.24      |
| replan to new goal     | 96     | 0.0551    | 21.14 | 2.24      |

“Five events. Clearance never drops below 2.0 and no bad state is ever entered, in any of them.”

## 4. What to point at in the other modes

**--tests** — every one of the six required cases ends in `optimality check : PASS (Dijkstra = x, D* Lite = x)`.

“Each result is verified against a completely separate Dijkstra implementation reached through the same abstract `Planner` interface. Two different algorithms agreeing on the cost is much stronger than one algorithm checking itself.”

Also worth flagging in test 2c: **0 expansions** when a bad state appears that the current path does not care about. Detecting “nothing to do” for free is a real property, not an accident.

**--sweep** — the route flips between  $\lambda = 1.50$  and  $1.75$ ; the closed form predicts  $\lambda^* = 1.511$ .

“But look at the note underneath. There is a third route in that graph, cost 3.9 and clearance 1.89, and **no value of  $\lambda$  ever selects it**. It is not dominated — nothing is both cheaper and safer. It just sits above the convex hull of the other two in the cost-penalty plane, and a linear objective can only ever return hull vertices. The score function in the assignment brief is linear, so this is a limitation of the objective we were given, not of the search. If you want that middle route you need a hard clearance constraint, not a better  $\lambda$ .”

**--bench** — 22,500 states, 178,808 transitions. 6,169 expansions with the heuristic, 22,380 with  $h = 0$ . Same planner, same heap, same weights; only the heuristic changes.

**--scaling** — the honest table.

“Localised changes:  $3.8\times$  up to  $67.8\times$  faster than replanning. But cut a third of the solution path and repair is *slower* than starting over — down to  $0.51\times$ . D\* Lite has to drive the invalidated g-values to infinity first and then re-expand the same region, so those states get processed twice. That is a known property of the algorithm, and the practical conclusion is

that a real system should measure how much of the tree a change invalidates and fall back to a fresh search past a threshold.”

Volunteering the case where your method loses is worth more than any of the wins.

## 5. Questions to expect

**Why D\* Lite over LPA\*?** LPA\* searches forward from a fixed start, so its g-values are start-relative and moving the agent discards all of them. D\* Lite searches backwards from the goal, so g-values are goal-relative and survive agent motion; the heuristic shift is absorbed by the  $k_m$  offset. Frame 2 of the demo is the whole argument: zero expansions after six steps.

**How do you guarantee no bad state is ever entered?** It is not a penalty. Any transition whose head or tail is a bad state has weight infinity, so no finite parameter setting can buy a path through one. That is why the “bad states visited” column is structurally zero rather than empirically zero.

**Is your heuristic admissible?**  $h(s) = \eta \|s - s_{start}\|$  with  $\eta$  the minimum cost-per-unit-length over all available transitions, computed from the graph rather than assumed. Since every effective weight is at least the nominal cost, any path costs at least  $\eta$  times its total length, which by the triangle inequality is at least  $\eta \|s - s_{start}\|$ . The same triangle inequality gives consistency, which is what D\* Lite actually needs.

**What happens when you insert a cheaper edge — does  $\eta$  change?** Yes, and that is a real trap. A cheaper edge lowers  $\eta$ , which lowers  $h$ , which makes every key already in the queue too large. D\* Lite’s built-in stale-key check only handles keys that are too small. The implementation detects the drop and re-keys the open list in  $O(|U|)$ . Test 6 hits this case: inserting the shortcut halves  $\eta$  from 1.0 to 0.5.

**Why is the score function not what you minimise?** Because the minimum-clearance term is a minimum over the whole path, not a sum over transitions, and shortest-path algorithms need additive weights. The planner minimises an additive surrogate and reports  $\text{Score}(P)$  for the plan it returns. The sweep shows the surrogate moves  $\text{Score}$  in the intended direction — and where it cannot.

**How would you handle a knowledge graph?** Entities become states, relations become transitions, and the embedding is whatever vector representation you already have. Two changes: an id map in the loader, since URIs are not consecutive integers, and  $\eta$  computed under the same metric as the embedding. Nothing in the search assumes the space is physical.

**What is the biggest weakness?** The clearance field rebuild is  $O(nkd)$  and dominates setup at scale — 23.5 ms against 8.8 ms of search at 40,000 states. A k-d tree would cut it to  $O(n \log k)$ , and since only states within  $d_{safe}$  of a changed hazard can be affected, a spatial index would also make bad-state updates local instead of global.

## 6. If something goes wrong

- **Grid frames wrap.** Widen the terminal or drop `--slow` and scroll back.
- **--demo scrolls past too fast.** Use `--slow`, or `./safe_planner --demo | less -R`.
- **Timing numbers differ from the report.** Expected on different hardware; the expansion counts are deterministic and will match exactly.
- **Asked to prove it is not hard-coded.** Edit `warehouse.graph` live — move a drum, change a cost — and re-run `--solve`. The optimality check re-verifies against Dijkstra on the spot.